Министерство образования Республики Беларусь
Учреждение образования
Белорусский Государственный университет информатики и радиоэлектроники

Инженерно-экономический факультет
Кафедра экономической информатики

Отчет

по лабораторной работе № 7

на тему «Обеспечение надёжного взаимодействия сервисов и оптимизация распределённой системы»

Выполнила: Абу Халава И.М.
студент группы 377901

Проверил: Сильванович Ю.В.

Минск 2026

Введение

Лабораторная работа № 7 посвящена надёжности взаимодействия, восстановлению событий и кешированию в программном средстве анализа рентабельности продукции. Система объединяет производственный и финансовый учёт, рассчитывает себестоимость и прибыль, предоставляет GraphQL API и отчётность. Исправления бизнес-логики и инфраструктуры собраны в итоговой исполняемой версии lab7_profitability_system; прежние каталоги лабораторных сохраняют историю этапов разработки.

Сервисы владеют отдельными базами данных. Gateway проверяет пользователя и права, сервисный слой выполняет бизнес-операции, репозитории сохраняют данные только своего сервиса. В итоговой версии синхронный обмен выполняется по gRPC, а асинхронные реакции — через Kafka.

1 Механизмы устойчивости

Компонент	Поведение
Читающие gRPC	Timeout каждой попытки; bounded retry временных ошибок
Изменяющие gRPC	Одна попытка; автоматический повтор после timeout исключён
Kafka producer	Подтверждение acks=all, ограниченные повторы
Kafka consumer	Ограниченные повторы handler; DLQ с подтверждением; явный commit
Outbox	Калькуляция, распределения и событие одной транзакцией; фоновая доставка
Redis	TTL контекста; инвалидация по MasterDataChanged; работа при отказе кеша
Логи и события	Структурированные поля и общий correlationId сценария

2 Безопасные повторы gRPC

JsonGrpcClient различает чтение и изменение состояния. Методы Get, List, Lookup и ProfitabilitySummary могут повторяться при UNAVAILABLE, DEADLINE_EXCEEDED, RESOURCE_EXHAUSTED и ABORTED. CalculateCost, CalculateProfitability, StartImport, RetryImport, CreateExport и ResolveExternalMapping получают max_attempts=1. Ошибки INVALID_ARGUMENT, NOT_FOUND и PERMISSION_DENIED не повторяются.

Если ответ на успешно сохранённый CalculateCost потерян, клиент получает ошибку с неопределённым результатом, но транспорт не создаёт вторую версию расчёта автоматическим повтором. Пользователь может проверить costBreakdown и статус операции перед явным повторным запуском. Таймаут сам по себе не доказывает откат серверной транзакции.

```
read_only = (method.startswith(("Get", "List", "Lookup"))
             or method == "ProfitabilitySummary")
policy = read_policy if read_only else RetryPolicy(max_attempts=1)
# Default: timeout=2s, read attempts=3, initial backoff=0.05s.
```

3 Kafka consumer и DLQ

Декодирование bytes и проверка envelope выполняются внутри цикла обработки. Некорректные JSON, тип payload, обязательные поля и версия схемы превращаются в контролируемую ошибку сообщения. Consumer сохраняет повреждённые bytes в base64 и направляет их в DLQ; повреждение одной записи не завершает consumer.

Handler повторяется ограниченное число раз. После исчерпания попыток публикация DLQ должна получить подтверждение брокера. Только handler success или DLQ success разрешает commit offset+1 для конкретной partition. При одновременном отказе обработки и DLQ commit отсутствует, текущая запись повторяется и последующие offsets не подтверждаются поверх неё.

```
while not stopped and not acknowledged:
    event = decode_inside_record_error_boundary(raw)
    acknowledged = process_with_retry(event) or confirmed_dlq(event)
    if acknowledged:
        commit_only_this_partition(record.offset + 1)
    else:
        wait_before_retry()
```

Этот фрагмент показывает условие подтверждения; реализация находится в consumer.py. Даже ошибка commit не считается завершением обработки: сохранённая Kafka позиция не продвигается, а повтор допускается по модели at-least-once. Поэтому обработчики используют upsert результатов и проекций.

4 Согласованность калькуляции и события

Прямая последовательность «сохранить калькуляцию → вызвать Kafka» заменена transactional outbox. CostingRepository.save_batch в одной транзакции сохраняет все COST_CALCULATION, OVERHEAD_ALLOCATION и OUTBOX_EVENT с полным EventEnvelope. Версии назначаются внутри транзакции. Если сохранение любого элемента не удалось, весь пакет откатывается.

```
BEGIN
  INSERT cost_calculations (..., source_versions, planned_unit_cost);
  INSERT overhead_allocations (..., production_order_id, cost_center_id);
  INSERT outbox_events (event_id, envelope, published_at=NULL);
COMMIT

# Background dispatcher:
for pending in outbox_events:
    publish_and_wait_for_broker_ack(pending.envelope)
    mark_published_only_after_ack(pending.event_id)
```

При отказе Kafka сохранённый расчёт остаётся корректным, а событие — pending с last_error. Фоновый dispatcher повторяет отправку после восстановления брокера и после перезапуска сервиса. В Kafka runtime запрос не ждёт доставки каждого события. Повторная отправка сохраняет тот же eventId и correlationId. Сбой между подтверждением Kafka и отметкой published_at может привести к дубликату, поэтому доставка заявлена как минимум один раз, без обещания exactly-once.

Такой же локальный outbox фиксирует итог импортов. Успешный импорт не превращается в ImportFailed только из-за недоступности брокера: реальное состояние загрузки и pending-событие уже сохранены. Analytics повторяемо сохраняет результат по calculation_id, Reporting — проекцию по продукту и периоду с идентификатором результата.

5 Сквозная прослеживаемость и версии

Все CostCalculationCompleted одного пакета получают общий correlationId. Analytics consumer передаёт исходный correlationId в calculate_for и далее в EventEnvelope.create для ProfitabilityCalculated. Reporting получает тот же идентификатор; логи handler и producer содержат correlation_id. Новый бизнес-сценарий получает новый UUID, но продолжение уже начатого события не создаёт независимую цепочку.

Analytics запрашивает точную version из CostCalculationCompleted. COST_CALCULATION хранит source_versions Master Data, Production и Financial, версию плана и planned_unit_cost. Изменение источника переводит зависимые результаты в INVALIDATED. Отсутствие плана выражено null, а не вычислением «плана» из факта.

6 Redis и инвалидация

Ключ costing-context:<product_id>:<period> хранит нормативные материалы, базу распределения и версию Master Data. TTL по умолчанию 300 секунд. При cache hit повторное сетевое чтение не требуется. MasterDataChanged удаляет кеш по префиксу и инвалидирует калькуляции; Analytics и Reporting исключают зависимые результаты. При недоступном Redis логируется ошибка, после чего выполняется исходный gRPC-запрос.

7 Архитектура и сценарий

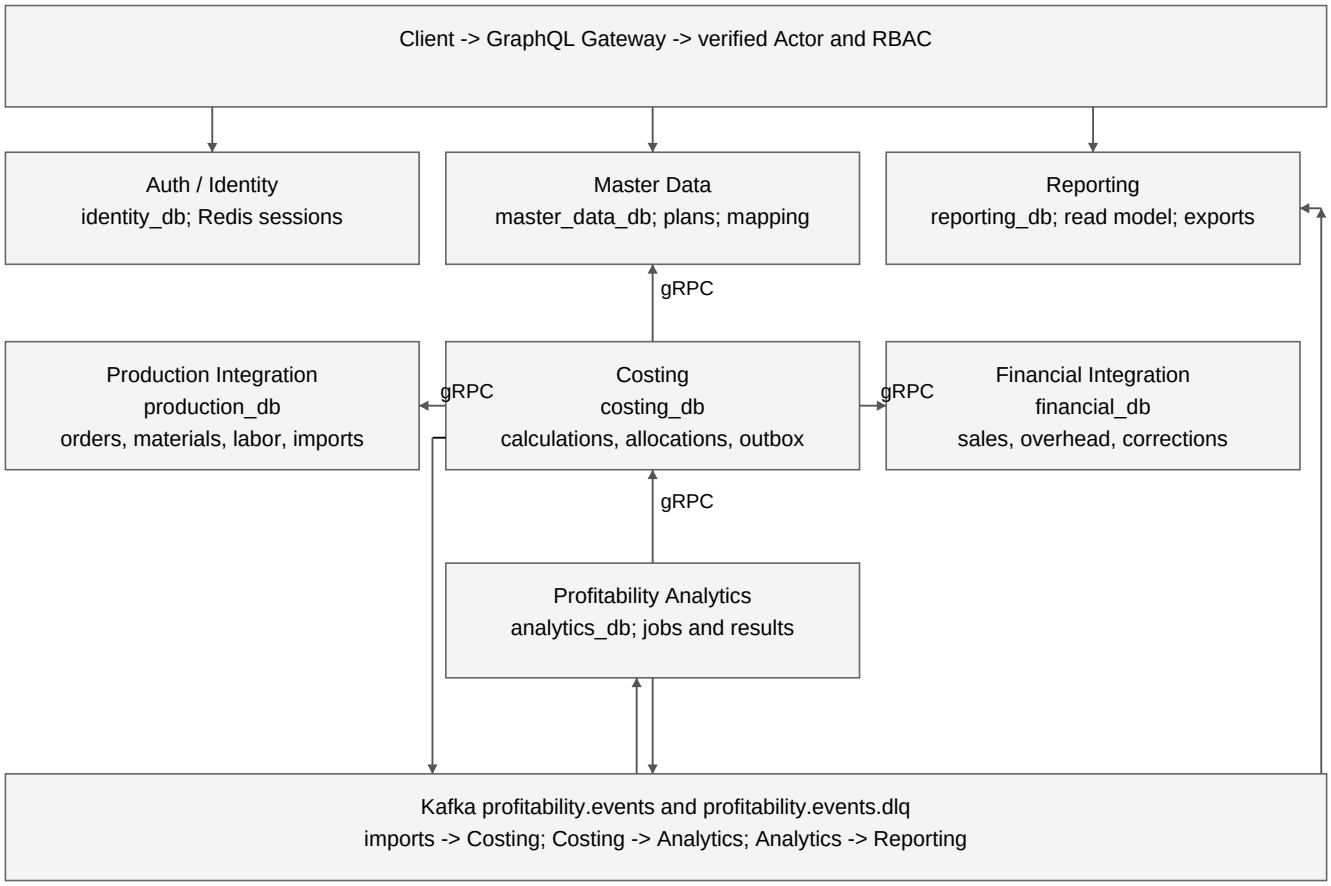


Рисунок. Сервисы, отдельные базы данных и асинхронное обновление Reporting

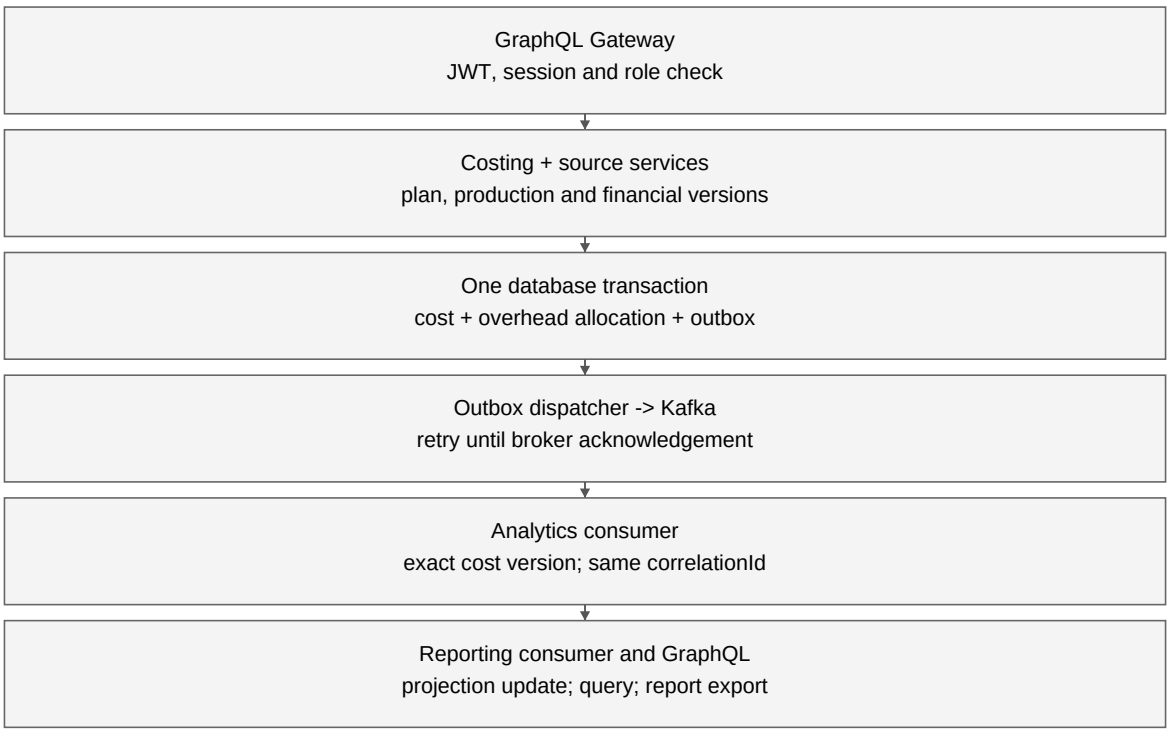


Рисунок. Последовательность расчёта и получения результатов

8 Логирование

Поле	Назначение
operation, target	Метод и адрес зависимого сервиса
attempt, duration_ms	Номер попытки и длительность
event_type, correlation_id	Событие и связь с бизнес-сценарием
error_type	Тип транспортной либо прикладной ошибки
last_error / published_at	Состояние восстановления доставки в outbox

Чувствительные служебные токены и пользовательские пароли в логи межсервисных операций не выводятся. Прямые RPC требуют служебный JWT и проверку роли; gRPC-порты не публикуются на хост. Безопасность пользовательского контура с JWT, Redis-сессиями и актуальными ролями сохранена.

9 Регрессионная проверка

Сценарий	Что проверяется
Ответ CalculateCost потерян	Изменяющий RPC вызывается один раз
Kafka недоступна после расчёта	Калькуляция и pending outbox сохранены; после восстановления событие доставляется
Handler и DLQ недоступны	Offset не подтверждается
Повреждённый JSON	Consumer остаётся жив; bytes доступны в DLQ; подтверждение только после handoff
Событие старой версии	Analytics использует указанную версию, не последнюю
Цепочка Costing → Analytics	ProfitabilityCalculated содержит исходный correlationId
Фильтр пустой группы	Список пуст; сводка нулевая
Импорт, экспорт и безопасность	Факты сохранены, файлы готовы, чужие отчёты и неавторизованные RPC запрещены

```
LAB7_EVENT_BACKEND=memory LAB7_SESSION_BACKEND=memory \
LAB7_CACHE_BACKEND=memory python -m pytest -q

# Runtime with Kafka and Redis:
docker compose build
docker compose run --rm auth python -m app.scripts.seed
docker compose up -d --wait
```

Набор unit/integration тестов проверяет реальные gRPC-вызовы, а Kafka producer/consumer и Redis cache в этих тестах могут использовать управляемые backends. Проверка настоящего брокера и Redis выполняется отдельно в Compose. Результат текущего прогона и команды воспроизведения сохранены в TESTING.md итогового проекта.

Заключение

Повторы транспортных операций учитывают их влияние на состояние. Событие калькуляции восстанавливается через outbox, повреждённые сообщения обрабатываются внутри consumer, а offset не теряется при отказе DLQ. Общий correlationId и сохранённые версии исходных данных связывают калькуляцию, аналитику и выдачу Reporting в проверяемый бизнес-сценарий.